LangGraph.py

Grafo de resolución con reintentos · código y diagrama de secuencia

Código

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

# 1. Estado: el mensaje que envia el usuario, si hay que escalarlo y cuantos intentos
class WriteState(TypedDict):
    messages: str
    is_escalated: bool
    attempts: int

# 2. Nodo escritor: genera/mejora el borrador (simulado: suma 3 puntos por vuelta)
def intentar_resolver(state: WriteState) -> dict:
    counter_attempts = state["attempts"] + 1
    print(f"[intentando resolver] revisión {state['attempts'] + 1} -> intentos
{counter_attempts}")
    return {
        "messages": f"Novedades sobre '{state['messages']}' (calidad {counter_attempts}/10)",
        "attempts": counter_attempts,
        "is_escalated": False,
    }

# 3. Nodo crítico: no modifica el estado; la decisión la toma la arista condicional
def escalar(state: WriteState) -> dict:
    return {"is_escalated": True}

# 4. Arista condicional: ¿aprobamos, nos rendimos o seguimos revisando?
def route(state: WriteState) -> str:
    if state["attempts"] >= 2:
        return "resuelto"
    #if state["time"] >= 2:        # tope para no iterar infinito
    #    return "no_resuelto"
    return "seguir_intentando_resolver"                    # vuelve al ciclo de intntar
resolver el problema del usuario

# 5. Construcción del grafo
g = StateGraph(WriteState)
g.add_node("resolver", intentar_resolver)
g.add_node("escalar", escalar)

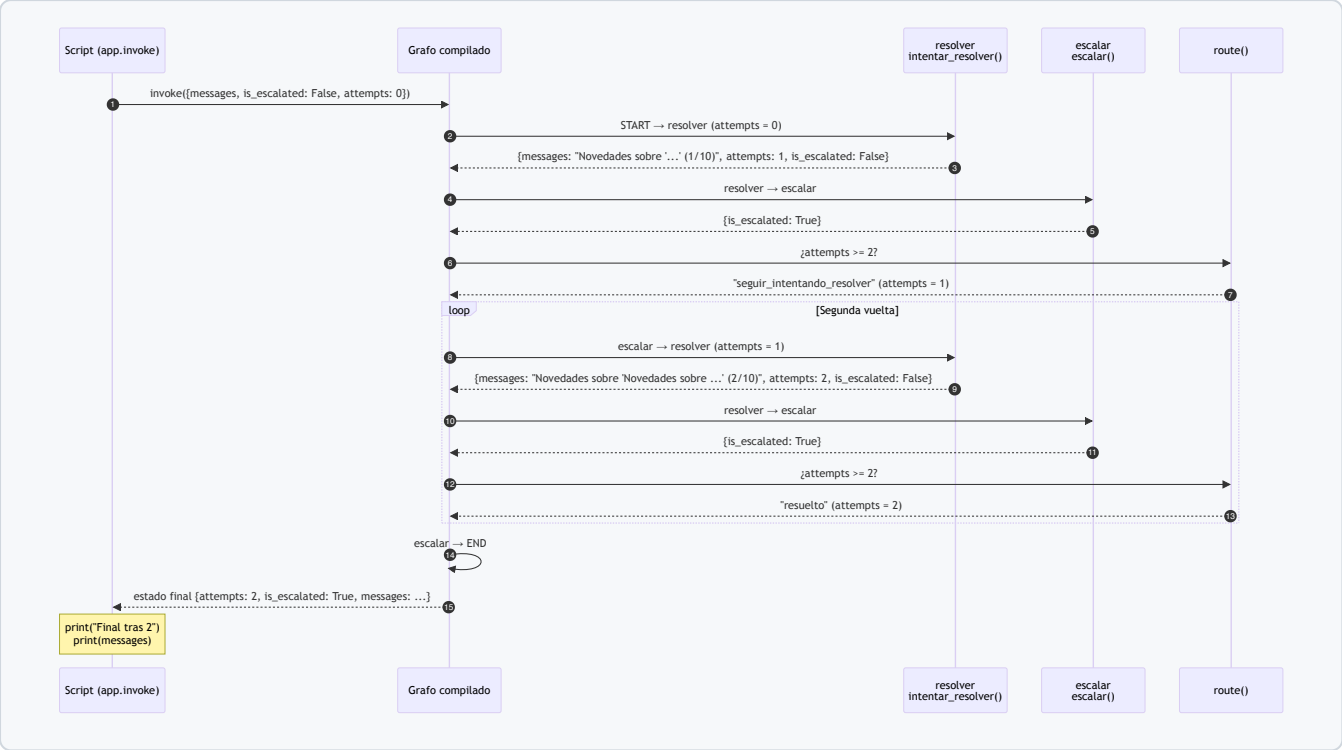
g.add_edge(START, "resolver")
g.add_edge("resolver", "escalar")
g.add_conditional_edges(
    "escalar",
    route,
    {"resuelto": END, "no_resuelto": END, "seguir_intentando_resolver": "resolver"},
)

app = g.compile()

# 6. Ejecución
final = app.invoke({"messages": "Perdi mi contraseña y no me deja resolver", "is_escalated":
False, "attempts": 0 })
print(f"Final tras {final['attempts']}")
```

```
print(final["messages"])
```

Diagrama de secuencia (ejecución con attempts = 0)



Salida esperada

```
[intentando resolver] revisión 1 -> intentos 1
[intentando resolver] revisión 2 -> intentos 2
Final tras 2
Novedades sobre 'Novedades sobre 'Perdi mi contraseña y no me deja resolver' (calidad 1/10)'
(calidad 2/10)
```

Observaciones

- **messages se anida:** resolver sobrescribe messages envolviendo el texto anterior en cada vuelta.
- **escalar corre siempre:** se ejecuta antes de route, así que is_escalated termina en True aunque el caso se "resuelva".
- **no_resuelto es una ruta muerta:** está en el mapa de rutas pero route nunca la devuelve.